Lehrstuhl für Software and Computational Systems
Henrik Wachowitz

Übungen zur Vorlesung
Softwaretechnik – Wintersemester 2025/2026

Übungsblatt 5

Zu bearbeiten bis Mittwoch, 19.11.

1 Funktionen

Hinweis: Sie finden den Code zu auch in der Aufgabenstellung in Moodle.

a) Betrachten Sie die folgende Funktion:

```
double calculateFinalPrice(double basePrice, boolean isMember) {
    double priceWithTax = basePrice + (basePrice * TAX_RATE);
    if (isMember) {
        return priceWithTax * MEMBER_DISCOUNT;
    } else {
        return priceWithTax;
    }
}
```

Listing 1: Preiskalkulationsfunktion

(oder in Python: Listing 12)

i) Das Argument `isMember` ist eine spezielle Art von boolean Argument. Wie nennt man diese? Es ist ein Flag-Argument. Es handelt sich um einen boolean der das Verhalten der Funktion beeinflusst.

ii) Führen Sie ein Refactoring durch, um das Argument `isMember` zu entfernen.

```
public double calculateFinalPriceForNonMembers(double
    basePrice) {
    return basePrice + (basePrice * TAX_RATE);
}

public double calculateFinalPriceForMembers(double
    basePrice) {
    double priceWithTax = basePrice + (basePrice *
        TAX_RATE);
    return priceWithTax * MEMBER_DISCOUNT;
}
```

Listing 2: Preiskalkulationsfunktionen

b) Betrachten Sie die folgende Funktion:

```
List<List<T>> chopUp(List<T> list, int size) {
    List<List<T>> result = new ArrayList<>();
    for (int nextListStart = 0; nextListStart < list.size();
        nextListStart += size) {
        int lastIndex = Math.min(nextListStart + size,
            list.size());
```

```
        result.add(list.subList(nextListStart, lastIndex));
    }
    return result;
}
```

Listing 3: Chop Up

(Python: Listing 14)

i) Was macht die Funktion? Die Funktion teilt eine Liste in mehrere kleinere Listen auf, wobei jede kleinere Liste eine maximale Größe von `size` hat.

ii) Der Name der Funktion ist nicht aussagekräftig. Geben Sie der Funktion einen Namen der direkt sagt, was sie tut. Ein besserer Name wäre z. B. `splitListIntoChunks`.

c) Beim refactoring einer Mathematikbibliothek stoßen sie auf folgenden Code:

```
record Point(int x, int y) {}

/**
 * Finds the intersection point of two lines defined by their endpoints.
 *
 * @param x1 x-coord of the first endpoint of the first line
 * @param y1 y-coord of the first endpoint of the first line
 * @param x2 x-coord of the second endpoint of the first line
 * @param y2 y-coord of the second endpoint of the first line
 * @param x3 x-coord of the first endpoint of the second line
 * @param y3 y-coord of the first endpoint of the second line
 * @param x4 x-coord of the second endpoint of the second line
 * @param y4 y-coord of the second endpoint of the second line
 * @return the intersection point of the two lines
 * @throws IllegalArgumentException if the lines are parallel and do not intersect
 */
Point findIntersectionOfLines(int x1, int y1, int x2, int y2,
                              int x3, int y3, int x4, int y4) {
// Function Body is not relevant for the exercise

}
```

Listing 4: Schnittpunkt zweier Geraden (Python s. Listing 17)

Führen sie ein Refactoring durch, um die Anzahl der Funktionsargumente zu reduzieren. Sie dürfen den Rumpf der Methode ignorieren. Sie dürfen weitere Klassen erstellen, wenn nötig. Schaffen sie es, die Anzahl der Argumente auf eins zu reduzieren, ohne andere Clean-Code-Prinzipien zu verletzen?

```
record Point(int x, int y) {}

class Line {
    private final Point start;
    private final Point end;

    public Line(Point start, Point end) {
        this.start = start;
        this.end = end;
    }
```

```java

    public Point findIntersectionWith(Line other) {
        // Function Body is not relevant for the exercise
    }
}
```

Listing 5: Refactored Schnittpunkt zweier Geraden

d) Betrachten Sie die folgende Funktion, die Studierendendaten aus einer Datei verarbeitet:

```java
record Student(String name, int age, String major) {}

List<Student> readStudentsFromFile(String filename) throws IOException {
    List<Student> students = new ArrayList<>();
    BufferedReader reader = new BufferedReader(new FileReader(filename));
    String line;
    while ((line = reader.readLine()) != null) {
        String[] parts = line.split(",");
        String name = parts[0];
        int age = Integer.parseInt(parts[1]);
        String major = parts[2];
        Student student = new Student(name, age, major);
        students.add(student);
    }
    reader.close();
    return students;
}
```

Listing 6: Verarbeitung von Studierendendaten (Python s. Listing 19)

i) Was macht die Funktion? Die Funktion liest eine Datei mit Studierendendaten ein, verarbeitet jede Zeile der Datei, um Studierendenobjekte zu erstellen, und speichert diese Objekte in einer Liste.

ii) Die Funktion bewegt sich nicht auf einer einheitlichen Abstraktionsebene. Identifizieren Sie verschiedene Abstraktionsebenen in der Funktion. Die Funktion mischt verschiedene Abstraktionsebenen:

- Hohe Abstraktionsebene: Datei öffnen, Daten verarbeiten, Ergebnisse speichern
- Mittlere Abstraktionsebene: Zeilen durchgehen, Studierendendaten erstellen
- Niedrige Abstraktionsebene: String-Parsing mit `split`, Array-Zugriffe, String-zu-Integer-Konvertierung

iii) Strukturieren Sie die Funktion in eine Abstraktionshierarchie um. Die abstrakteste Funktion sollte als Um-zu-Text lesbar sein (z. B. „Um Studierendendaten zu verarbeiten, öffne die Datei, um dann jede Zeile zu verarbeiten.“).

```java
List<Student> readStudentsFromFile(String filename) throws IOException {
    List<Student> students = new ArrayList<>();
    BufferedReader reader = new BufferedReader(new FileReader(filename));
    String line;
    while ((line = reader.readLine()) != null) {
```

```
        Student student = parseStudentFromLine(line);
        students.add(student);
    }
    reader.close();
    return students;
}

private Student parseStudentFromLine(String line) {
    String[] parts = line.split(",");
    String name = parts[0];
    int age = Integer.parseInt(parts[1]);
    String major = parts[2];
    return new Student(name, age, major);
}
```

Listing 7: Strukturierte Abstraktionshierarchie

Erklärung: Die Funktion `processStudentData` ist nun auf der höchsten Abstraktionsebene und liest sich als Um-zu-Text: „Um Studierendendaten zu verarbeiten, lies alle Zeilen, um dann für jede Zeile einen Studierenden zu erstellen, um dann die Studierenden in einer Liste zu speichern“. Jede aufgerufene Funktion befindet sich auf der nächstniedrigeren Abstraktionsebene.

2 Dyadische Methoden

Betrachten Sie die in Unittests häufig verwendete Methode `assertEquals` (siehe Listing 8). Da es sich hierbei um eine Bibliotheksmethode handelt, können wir sie nicht direkt verändern.

```
testSomething() {
    //... test setup code ...
    assertEquals(expectedValue, actualValue);
}
```

Listing 8: Dyadische Methode assertEquals (Python s. Listing 15)

a) Was ist das Problem bei der Methodensignatur von `assertEquals`? Die beiden Argumente sind vom gleichen Typ und die Reihenfolge ist nicht durch den Kontext klar ersichtlich. Dadurch können sie leicht verwechselt werden.

b) Verwenden Sie die Strategie „*extraktion einer neuen Klasse*“ um die dyadische Methode zu vermeiden. Schreiben Sie den Code für die neue Klasse und die veränderte Testmethode (Antwort in Python ist erlaubt). **Hinweis:** Sie dürfen die Methode `assertEquals` in ihrer Klasse verwenden.

```
public class ExpectedValue {
    private final Object expected;

    public ExpectedValue(Object expected) {
        this.expected = expected;
    }

    public void assertEquals(Object actual) {
        assertEquals(expected, actual);
    }
}
```

```java

public void testSomething() {
    // ... test setup code ...
    new ExpectedValue(expectedValue).assertEquals(actualValue);
}
```

Listing 9: Vermeidung dyadischer Methode durch Extraktion einer neuen Klasse

3 Klassen: Dependency Injection

Betrachten Sie die folgenden Klassen.

```java
interface OutputWriter {
    void print(String message);
}

class ConsoleWriter implements OutputWriter {
    void print(String message) {
        System.out.println(message);
    }
}

class User {
    private final OutputWriter outputWriter;
    private final String name;

    User(String name) {
        outputWriter = new ConsoleWriter();
        this.name = name;
    }

    void greet() {
        outputWriter.print("Hello, " + name + "!");
    }
}

class TestUser {
    void testGreet() {
        // ???
    }
}
```

Listing 10: Benutzerklasse mit Abhängigkeit (Python Listing 21)

a) Wie würden Sie in der aktuellen Form automatisiert testen, dass ein User „Alice“ tatsächlich mit `Hello, Alice!` grüßt? Testen ist sehr schwierig, da die Klasse `User` direkt eine Instanz von `OutputWriter` erstellt. Man müsste den `System.out` Stream umleiten, um die Ausgabe zu überprüfen, was umständlich und fehleranfällig ist.

b) Benutzen Sie *Dependency Injection*, um die Klasse so umzuschreiben, dass sie leicht testbar ist. Schreiben Sie anschließend einen Test, der überprüft, dass ein User „Alice“ tatsächlich mit `Hello, Alice!` grüßt.

```java
class User {
    private final OutputWriter outputWriter;
    private final String name;

    User(String name, OutputWriter outputWriter) {
        this.outputWriter = outputWriter;
        this.name = name;
    }

    void greet() {
        outputWriter.print("Hello, " + name + "!");
    }
}

class MockingWriter implements OutputWriter {
    private String lastMessage;

    void print(String message) {
        lastMessage = message;
    }

    void assertLastMessageEquals(String expected) {
        assertEquals(expected, lastMessage);
    }
}

class TestUser {
    void testGreet() {
        MockingWriter mockWriter = new MockingWriter();
        User user = new User("Alice", mockWriter);
        user.greet();
        mockWriter.assertLastMessageEquals("Hello, Alice!");
    }
}
```

Listing 11: Benutzerklasse mit Dependency Injection

Appendix

```python
def calculate_final_price(base_price: float, is_member: bool) -> float:
    price_with_tax = base_price + (base_price * TAX_RATE)
    if is_member:
        return price_with_tax * MEMBER_DISCOUNT
    else:
        return price_with_tax
```

Listing 12: Preiskalkulationsfunktion

```python
def calculate_final_price_for_non_members(base_price: float) -> float:
    return base_price + (base_price * TAX_RATE)

```

```python
def calculate_final_price_for_members(base_price: float) -> float:
    price_with_tax = base_price + (base_price * TAX_RATE)
    return price_with_tax * MEMBER_DISCOUNT
```

Listing 13: Preiskalkulationsfunktionen

```python
def chop_up(lst: list, size: int) -> list[list]:
    result = []
    for next_list_start in range(0, len(lst), size):
        last_index = min(next_list_start + size, len(lst))
        result.append(lst[next_list_start:last_index])
    return result
```

Listing 14: Chop Up

Ein besserer Name wäre z. B. `split_list_into_chunks`.

```python
def test_something():
      # ... test setup code ...
      assert_equal(expected_value, actual_value)
```

Listing 15: Dyadische Methode assertEquals

```python
class ExpectedValue():
    def __init__(self, expected):
        self.expected = expected

    def assert_equals(self, actual):
        assert_equals(self.expected, actual)

def test_something():
    # ... test setup code ...
    ExpectedValue(expected_value).assert_equals(actual_value)
```

Listing 16: Vermeidung dyadischer Methode durch Extraktion einer neuen Klasse

```python
def find_intersection_of_lines(x1: int, y1: int, x2: int, y2: int,
                               x3: int, y3: int, x4: int, y4: int)
                                  -> Point:
    """
    Finds the intersection point of two lines defined by their
       endpoints.

    Args:
        x1: x-coordinate of the first endpoint of the first line
        y1: y-coordinate of the first endpoint of the first line
        x2: x-coordinate of the second endpoint of the first line
        y2: y-coordinate of the second endpoint of the first line
        x3: x-coordinate of the first endpoint of the second line
        y3: y-coordinate of the first endpoint of the second line
        x4: x-coordinate of the second endpoint of the second line
        y4: y-coordinate of the second endpoint of the second line

    Returns:
       A Point representing the intersection point if the lines
          intersect.

```

```
    Raises:
        ValueError: If the lines are parallel and do not intersect.
    """
    # Function Body is not relevant for the exercise
    pass
```

Listing 17: Schnittpunkt zweier Geraden

```
class Point:
    x: int
    y: int

class Line:
    start: Point
    end: Point

    def __init__(self, start: Point, end: Point):
        self.start = start
        self.end = end

    def find_intersection_with(self, other: 'Line') -> Point:
        """
        Finds the intersection point of this line with another line.

        Args:
            other: The other line to find the intersection with.

        Returns:
            A Point representing the intersection point if the lines intersect.

        Raises:
            ValueError: If the lines are parallel and do not intersect.
        """
        # Function Body is not relevant for the exercise
        pass
```

Listing 18: Refactored Schnittpunkt zweier Geraden

```
@dataclass(frozen=True)
class Student:
    name: str
    age: int
    major: str

def read_students_from_file(filename: str) -> list[Student]:
    from pathlib import Path
    students: list[Student] = []
    with Path(filename).open(encoding="utf-8") as f:
        for line in f:
            parts = line.strip().split(",")
            name = parts[0]
            age = int(parts[1])
            major = parts[2]
```

```
            student = Student(name, age, major)
            students.append(student)
    return students
```

Listing 19: Verarbeitung von Studierendendaten

```
@dataclass(frozen=True)
class Student:
    name: str
    age: int
    major: str

def parse_student_from_line(line: str) -> Student:
    parts = line.strip().split(",")
    name = parts[0]
    age = int(parts[1])
    major = parts[2]
    return Student(name, age, major)

def read_students_from_file(filename: str) -> list[Student]:
    from pathlib import Path
    students: list[Student] = []
    with Path(filename).open(encoding="utf-8") as f:
        for line in f:
            student = parse_student_from_line(line)
            students.append(student)
    return students
```

Listing 20: Strukturierte Abstraktionshierarchie

```
class ConsoleWriter:
    def print(self, message: str):
        print(message)

class User:
    def __init__(self, name: str):
        self.name = name
        self.output_writer = ConsoleWriter()

    def greet(self):
        self.output_writer.print(f"Hello, {self.name}!")

class TestUser:
    def test_greet(self):
        # ???
        ...
```

Listing 21: Benutzerklasse mit Abhängigkeit

```
class OutputWriter(Protocol):
    """
    A Protocol is a hint for static type checkers in Python.
    With this, we can define the expected interface for output
       writers.
    We can use the protocol to tell static type checkers
    and other developers what methods must be present when passing
```

```
    an object as dependency.
    """
    def print(self, message: str):
        pass

class MockingWriter:
    def __init__(self):
        self.last_message = None

    def print(self, message: str):
        self.last_message = message

    def assert_last_message_equals(self, expected_message: str):
        assert_equals(expected_message, self.last_message)

class User:
    def __init__(self, name: str, output_writer: OutputWriter):
        self.name = name
        self.output_writer = output_writer

    def greet(self):
        self.output_writer.print(f"Hello, {self.name}!")


class TestUser:
    def test_greet(self):
        mock_writer = MockingWriter()
        user = User("Alice", mock_writer)
        user.greet()
        mock_writer.assert_last_message_equals("Hello, Alice!")
```

Listing 22: Benutzerklasse mit Dependency Injection

Allgemeine Hinweise:

- Die vorgestellte Lösung ist lediglich ein Lösungsvorschlag. Es wird kein Anspruch auf Vollständigkeit oder Korrektheit erhoben.
- Das Blatt bereitet Sie auf die Vorlesung am **Mittwoch, 19.11.** vor.
- Das Blatt wird in der Übung am **20.11.** besprochen.
- Wir empfehlen Ihnen auch das Quiz in Moodle zu bearbeiten.